

National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R6 Sorting

Problems:

1. Consider the following implementation of the *insertion sort* algorithm:

```
def insert_cmp(x, xs, cmp):
    return (llist(x)
            if is_none(xs)
            else pair(x, xs)
            if cmp(x, head(xs))
            else pair(head(xs), insert_cmp(x, tail(xs), cmp)))

def insertion_sort_cmp(xs, cmp):
    return (xs
            if is_none(xs)
            else insert_cmp(head(xs),
                            insertion_sort_cmp(tail(xs), cmp),
                            cmp))
```

Given a llist of numbers, `xs`, as input, for each of the following, how would you call `insertion_sort_cmp` to get the required result?

We shall use the following llist `xs` as the example input:

```
xs = llist(6, 3, 8, 5, 1, 9, 6, 4, 2, 7)
```

- (a) The numbers in the result llist are in ascending order. For example,

```
insertion_sort_cmp(xs, ???)
# Result: llist(1, 2, 3, 4, 5, 6, 6, 7, 8, 9)
```

- (b) The numbers in the result llist are in descending order. For example,

```
insertion_sort_cmp(xs, ???)
# Result: llist(9, 8, 7, 6, 6, 5, 4, 3, 2, 1)
```

- (c) The result llist is the reverse of the input llist. For example,

```
insertion_sort_cmp(xs, ???)
# Result: llist(7, 2, 4, 6, 9, 1, 5, 8, 3, 6)
```

- (d) In the result llist, all the even numbers are placed before the odd numbers, and the even numbers are in ascending order, whereas the odd numbers are in descending order. For example,

```
insertion_sort_cmp(xs, ???)
# Result: llist(2, 4, 6, 6, 8, 9, 7, 5, 3, 1)
```

2. Consider the partial implementation of `merge_sort` shown in class:

```
# half, rounded downwards
def middle(n):
    ...

# put the first n elements of xs into a llist
def take(xs, n):
    ...

# drop the first n elements from the llist and return the rest
def drop(xs, n):
    ...

# merge two sorted llists into one sorted llist
def merge(xs, ys):
    if is_none(xs):
        return ys
    elif is_none(ys):
        return xs
    else:
        x = head(xs)
        y = head(ys)
        return (pair(x, merge(tail(xs), ys))
                if x < y
                else pair(y, merge(xs, tail(ys))))

def merge_sort(xs):
    if is_none(xs) or is_none(tail(xs)):
        return xs
    else:
        mid = middle(length(xs))
        return merge(merge_sort(take(xs, mid)),
                      merge_sort(drop(xs, mid)))
```

- (a) Use big- O , Θ or Ω notation to characterize the runtime of function `merge`, for the input llists `xs` and `ys` of the same length n .
- (b) Given the runtime of the functions `middle`, `take` and `drop` have the following order of growth:
- `middle` — $\Theta(1)$
 - `take` — $\Theta(n)$, where n is the value of the argument n
 - `drop` — $\Theta(n)$, where n is the value of the argument n

Characterize the runtime of `merge_sort`, using the big- O , Θ or Ω notation.